

Linear Regression

למה לעשות linear regression כשיש היום deep learning? כי מחלקת ההיפותזות יותר קטנה, וככה הסיכוי לעשות overfitting יותר קטן. אם אפשר לפתור עם מודל פשוט זה יותר טוב.

רגרסיה לינארית היא הבסיס של למידת מכונה, ועוזרת להבין את הרעיונות היותר מורכבים.

אנחנו מתמקדים בבעיית רגרסיה¹:

- יש dataset: $\{(x_i, y_i)\}_{i=1}^n$ כאשר $x_i \in \mathbb{R}^d, y_i \in \mathbb{R}$
- מקבלים n דוגמאות אימון $\mathcal{D} = \{(x_1, y_1), \dots, (x_n, y_n)\}$ כאשר לכל $i, x_i \in \mathbb{R}^d$, $y_i \in \mathbb{R}$ הוא ווקטור של features והוא continous target.

מחפשים ווקטור w וסקלר b (נקרא bias) עבורם:

$$\hat{y}_i = f(x_i) = w^T x_i + b$$

כאשר $w \in \mathbb{R}^d, b \in \mathbb{R}$ הם מקדמי המודל.

הערה: חלק מהפיצ'רים יכולים להיות לא מספריים. אם הם בינאריים אז אין בעיה - ממירים ל 0 או 1 - אבל אם יש למשל 3 אפשרויות, לא בטוח שהמרה ל 0,1,2 (או 0,0.5,1) תהיה נכונה. אפשר "לפצל" את הפיצ'ר הזה לכמה פיצ'רים בינאריים, אבל זה מגדיל את המימד.

כדי לפשט את המשוואה, נרצה להכניס את b לתוך w . אפשר לעשות עת זה באמצעות הוספת עמודת 1ים - כלומר $x_i \in \mathbb{R}^{d+1}$ כאשר הערך הראשון תמיד 1 - ואז גם $w \in \mathbb{R}^{d+1}$ ו $w_0 = b$ ככה המודל מפושט ל

$$\hat{y}_i = w^T x_i$$

Mean Squared Error Loss

בשביל למצוא רגרסיה הכי קרובה נשתמש בממוצע של ריבועי השגיאות:

$$MSE := \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \frac{1}{n} (y - Xw)^T (y - Xw)$$

רוצים למצוא w שממזער את MSE:

$$\hat{w} = \arg \min_w MSE(w)$$

מכיוון שה-MSE היא פונקציה ריבועים קמורה של w , יש לה מינימום גלובלי יחיד. נמצא אותו באמצעות הגרדיאנט:

$$\nabla_w MSE = 0$$

¹כלומר התיוגים הם מספרים

זהו Linear Regression - נקרא גם Ordinary Least Squares.
אם גוזרים מוצאים שהגרדיאנט הוא

$$\nabla_w MSE = -\frac{2}{n} X^T (y - Xw)$$

והוא מתאפס כאשר

$$w = (X^T X)^{-1} X^T y$$

זה אומר שאם יש סט חדש של data, $X_{new} \in \mathbb{R}^{m \times d}$, אפשר לחזות בקלות:

$$Y_{new} = X_{new} (X^T X)^{-1} X^T y$$

משמעות גיאומטרית

אם יש לנו תחזיות $\hat{y}$ ווקטור השגיאה הוא $(y - \hat{y})$, השגיאה היא אורתוגונלית לתחזיות:

$$(y - \hat{y})^T \hat{y} = \dots = 0$$

זה אומר שהתחזית שלנו היא הנקודה הכי קרובה ל y על המישור המוגדר ע"י X .

רגרסיה פולינומית

אם הקשר שאנחנו מחפשים הוא לא לינארי, נוכל עדיין להשתמש ברגרסיה לינארית ע"י שינוי data.

עבור $d = 1$ (כלומר $x \in \mathbb{R}$), נרצה למצוא פולינום ממעלה k :

$$\hat{y} = w_0 + x_1 x^1 + w_2 x^2 + \dots + w_k x^k$$

עבור n דוגמאות:

$$X = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^k \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^k \end{bmatrix}$$

אפשר לחשב:

$$\hat{w} = (X^T X)^{-1} X^T y$$

מתי קיים פתרון

הפתרון קיים אם המטריצה $(X^T X)$ הפיכה. מתי היא לא הפיכה?

- אם $d > n$ (יותר פיצ'רים מאשר דוגמאות)
 - אם יש תלות לינארית בין הפיצ'רים
- אם היא לא הפיכה - מה אפשר לעשות? אפשר להוסיף רגולריזציה.

Ridge Regression

נוסיף עוד מרכיב - penalty (עם הפרפרמטר $\lambda > 0$):

$$\hat{w}_{ridge} = \arg \min_w \left[(y - Xw)^T (y - Xw) + \lambda |w|_2^2 \right]$$

עם מחשבים את ה MSE בתוספת ה ridge regression מקבלים:

$$\hat{w}_{ridge} = (X^T X + \lambda I)^{-1} X^T y$$

למה זה עובד?

- אם (u, v) הם eigenpair של $X^T X$, אזי $(u + \lambda, v)$ הם eigenpair של $(X^T X + \lambda I)$.
- מכיוון ש u לא שלילי, ו λ חיובי, אז כל הערכים העצמיים חיוביים - והמטריצה הפיכה.

SVD perspective

יהי $X = U D V^T$ ה singular value decomposition של X .

$$\hat{y}_{ridge} = X (X^T X + \lambda I)^{-1} X^T y$$

כלומר ה ridge regression לוקח את הנקודות, עושה להם טרנספורמציה ל X , עושה scaling שנובע מהערכים העצמיים, ועושה טרנספורמציה חזרה ל y .

u^2 ערך עצמי ו v הווקטור העצמי שלו

קלאסיפיקציה בינארית

יש לנו סט דוגמאות $D = \{(x_1, y_1), \dots, (x_n, y_n)\}$, כאשר $x_i \in \mathbb{R}^d, y_i \in \{-1, 1\}$.
ורוצים למצוא היפותזה $h: \mathbb{R}^d \rightarrow \{-1, 1\}$.

פרספטרום

הרעיון הוא לקחת סט של משקולות על הפרמטרים, ולהכפיל אותם ווקטורית ב:

$$h_{w,b}(x) = \begin{cases} +1 & w^T x > b \\ -1 & w^T x < b \end{cases} = \text{sign} \left(\left(\sum_{i=1}^d w_i x_i \right) - b \right)$$

- decision boundary - $w^T x = 0$ - מייצג קו ישר שנקרא Linear Classifier.
- הוא עובד בצורה מדויקת רק כאשר data הוא linearly separable.

איך מוצאים את המשקולות? רוצים w שיקיים:

- $w^T x_i > 0$ כאשר $y_i = +1$
- $w^T x_i < 0$ כאשר $y_i = -1$
- בצורה קומפקטית:

$$\forall_{i=1, \dots, n} y_i (w^T x_i) > 0$$

זה לא משהו שאפשר למצוא בקלות באמצעות חישוב, אבל יש אלגוריתם שמתכנס:

- מתחילים עם w אקראי (או בוחרים שרירותית $w = 0$)
- כל עוד לא כל הדוגמאות מסווגות בצורה נכונה:

- לכל $i = 1, \dots, n$:

$$\hat{y}_i = \text{sign}(w^T x_i) \quad *$$

$$\text{אם } y_i \neq \hat{y}_i \quad *$$

$$w \leftarrow w + y_i x_i \quad .$$

כלומר - מוסיפים ל- w את הנקודה שסווגה לא נכון, כפול התגית.

מובטח לנו שזה יתכנס. זה עובד כי הוא מתקן את הזווית של classifier. אבל לא תמיד יש פתרון למשל עבור XOR. כלומר פרספטרום עובד רק אם הנתונים הן linearly separable.

כדי להתגבר על זה, צריך או להפוך את data לlinearly separable - או להשתמש ב loss function שיועד להתמודד עם קצת שגיאה.

Gradient Descent

אם פונקציית loss גזירה, אפשר למצוא את הפרמטרים שימזערו אותה באמצעות ירידה בגרדיאנט:

$$w \leftarrow w - \eta \frac{\partial \text{loss}}{\partial w}(w)$$

כלומר - אם אין לנו פתרון אנליטי, אפשר לבחור נקודה, למצוא את השיפוע, ללכת לפי כיוון ירידת השיפוע, וכן הלאה עד שמגיעים (קרוב מספיק) לשיפוע אפס.
GD עובד כאשר:

- פונקציית loss קמורה.
 - גודל הצעד קטן - $0 < \eta < \frac{1}{L}$, כאשר L הוא קבוע ליפשיץ של הגרדיאנט של loss.
- כלומר: loss לא "משתגע"

GD יקר לחישוב, כי הוא צריך לעבור על כל הדוגמאות, הרבה פעמים. לכן בפועל עושים:

(SGD) Stochastic Gradient Descent

במקום לחשב על כל הנקודות, מחשבים - בכל צעד - רק על חלק מהנקודות.
זה הרבה יותר מהיר - אבל גם מאוד כאוטי, וגם לא מנצל נכון את יכולות החישוב המקבילי של GPU.
לכן נרצה לעבוד עם batch - בוחרים בכל צעד תת־קבוצה אקראית של הנקודות.

- נשים לב: בGD רגיל loss תמיד ירד, ולכן אפשר לקבוע תנאי עצירה $|L_t - L_{t-1}| > \epsilon$
- בSGD אמנם נתקרב למינימום, אבל כשנגיע קרוב עליו נעשה כלהזמן קפיצות מסביבו ולא נתכנס.

כדי לגרום לSGD להתכנס, נקטין בכל פעם את גודל הצעד.

חזרה לbinary classification

נסתכל על התחזיות שלנו $w^T x_i$. רוצים $y_i (w^T x_i) > 0$. השם של הערך הזה הוא margin, והוא אומר לנו כמה אנחנו רחוקים מהקו הנכון - ואנחנו רוצים להיות רחוקים (בכיוון הנכון!) פונקציית loss היא:

$$l_{0-1}(w; x, y) = \begin{cases} +1 & yw^T x < 0 \\ 0 & yw^T x > 0 \end{cases}$$

רוצים למזער את השגיאה:

$$Err_w = \sum_{i=1}^n l_{0-1}(w; x_i, y_i)$$

הבעיה - l_{0-1} לא רציפה!

אפשרות אחת לפתור את זה עם hinge loss:

$$l_{hinge}(w; x, y) = \begin{cases} -yw^T x & yw^T x < 0 \\ 0 & yw^T x > 0 \end{cases}$$

כלומר הגרדיאנט של זה מתקן אם "הקלסיפיקציה לא נכונה". ואז אפשר לעדכן בכל צעד (דוגמה - SGD)

$$w \leftarrow w + \eta y_i x_i$$

שזה כמו בפרספטרון, רק ששם לא היה גודל צעד (כלומר תמיד היה $\eta = 1$).

קלסיפיקציה בינארית עם Logistic Regression

- במקום החלטה ± 1 , נשתמש בהסתברות $P(y = 1|x)$.
- במקום sign נשתמש ב"סיגמואיד":

$$\sigma(w^T x) = \frac{1}{1 + e^{-w^T x}}$$

זה נותן "ניורון" שאם הוא מעל $\frac{1}{2}$ הדוגמה חיובית ואם מתחת ל $\frac{1}{2}$ הדוגמה שלילית. מכיוון שעברנו לעולם של הסתברויות, נרצה למקסם את ההסתברות של datan בהתאם למודל:

$$P(y = 1|x) = \sigma(w^T x) \quad P(y = 0|x) = 1 - \sigma(w^T x)$$

ועבור כל datan training:

$$\mathcal{L}(w) = \prod_{y_i=1} P(y = 1|x_i) \prod_{y_i=0} P(y = 0|x_i)$$

שזה יוצא

$$\mathcal{L}(w) = \prod_{i=1 \dots n} \sigma(w^T x_i)^{y_i} \prod_{i=1 \dots n} (1 - \sigma(w^T x_i))^{1-y_i}$$

ומוציאים log כדי להפוך את זה לסכום:

$$\log \mathcal{L} = \sum_i y_i \log [\sigma(w^T (y = 0|x_i))] + \sum_i (1 - y_i) \log [1 - \sigma(w^T (y = 0|x_i))]$$

אנחנו רוצים למצוא מינימום, לא מקסימום, לכן נכפיל ב-1. נחשב את הגרדיאנט של השגיאה ונמצא שהוא

$$\nabla Err = \frac{1}{2} \sum_i x_i [y_i - \hat{y}_i]$$

וכלל העדכון:

$$w_{t+1} = w_t - \eta \sum_{i=1}^n x_i [\sigma(w_t^T x_i) - y_i]$$

אנחנו עדיין מקבלים סיווג לינארי - אבל מקבלים גם מרחק מהקו. נקודות שמאוד קרובות לקו הן כאלו שאנחנו פחות בטוחים בהן, אבל בגלל הצורה של הסיגמואיד ברגע שמתרחקים מרחק מסויים מהקו מגיעים קרוב מאוד לרמת הביטחון המקסימלית.